# Scrum Poker – Leitfaden fürs Fachgespräch

Im Fachgespräch wird **jedes Gruppenmitglied einzeln** zu beliebigen Stellen im Code befragt. Wer nichts erklären kann, bekommt die Note 1.0, egal wie gut die App ist. Die App wurde grösstenteils mit KI erstellt (Dokumentation, Kapitel 15 «KI-Einsatz»). Danach wurde sie auf Wunsch der Gruppe zweimal umgebaut: zuerst **vereinfacht**, dann an die **Kursfolien und die Projektvorlage** angeglichen. Sie benutzt jetzt fast nur noch, was in den Vorlesungen 1–9 vorkommt, jede Datei beginnt mit einem deutschen Kommentar, und fast jede Funktion ist kurz genug, um sie in einer Minute zu erklären. Ihr müsst den Code so gut kennen, als hättet ihr ihn selbst geschrieben. Dieser Leitfaden zeigt, wie.

### So arbeitet ihr mit diesem Leitfaden

1. **Teil A (Rundgang)** mit dem Code daneben lesen. Jede Datei öffnen und die genannten Zeilen ansehen. Zuerst den eigenen Bereich (siehe «Aufteilung für die Gruppe»), dann alles andere.
2. **Teil B (Musterantworten)** laut üben, **in eigenen Worten**, ohne abzulesen. Eine Person fragt, die andere zeigt die Stelle im Code und erklärt.
3. **Teil C (Fragenkatalog)** als Quiz verwenden.
4. **Teil E (Übungen)** wirklich durchführen: Code ändern, beobachten, zurücksetzen. Wer etwas selbst kaputt gemacht und repariert hat, kann es erklären.
5. **Teil D (Demo und Checkliste)** am Vortag einmal komplett durchspielen.

Verweise wie `PokerRoom.tsx:145-152` bedeuten Datei und Zeilennummer. Gibt es einen Dateinamen zweimal, steht der Ordner davor: `api/sessions.ts` (Client) und `routes/sessions.ts` (Server), `types/poker.ts` und `utils/poker.ts`. Pfade sind relativ zum Projektordner (im Code-ZIP der Ordner `scrum-poker/`). Im Editor springt ihr mit `Ctrl+G` (VS Code: «Go to Line») direkt zur Zeile.

### Lernplan: in vier Abenden bereit

| Abend | Was | Ziel am Ende |
|---|---|---|
| 1 | App starten (zwei Terminals, Teil D.2), zwei Browserfenster, alles einmal durchklicken. Dann «Das Wichtigste in fünf Sätzen» (A.0) und den **eigenen Bereich** in Teil A lesen, die passenden Foliensätze daneben. | Ich kann «meine» Dateien Zeile für Zeile vorlesen und sagen, was passiert. |
| 2 | Die zwei **anderen Bereiche** in Teil A. Teil B.1–B.5 laut üben. | Ich kann jede Datei in einem Satz beschreiben und den Klick auf eine Karte vom Button bis zum Server verfolgen. |
| 3 | Teil B.6–B.15 und Teil C zu zweit als Quiz. Übungen E.1–E.10. | Ich beantworte die Fragen ohne Unterlagen und habe Code geändert und zurückgesetzt. |
| 4 | Restliche Übungen, Demo (Teil D) mit Stoppuhr, gegenseitig zu zufälligen Zeilen befragen. | Ich kann jede beliebige Stelle zeigen und erklären. |

**Foliensätze**, auf die die Spalte «Folien» und die Angaben wie «09/36» (Foliensatz 9, Folie 36) verweisen:

| Nr. | Vorlesung | Was davon im Projekt vorkommt |
|---|---|---|
| **02** | Architektur und Technologien | Client und Server, HTTP-Methoden und Statuscodes, DevTools (Network, Application) |

| Nr. | Vorlesung | Was davon im Projekt vorkommt |
|---|---|---|
| **03** | Node.js | `package.json` mit `scripts` und `dependencies`, pnpm, ein Projekt = ein `package.json` |
| **04** | HTML und CSS Basics | semantische Elemente (`main`, `section`), Selektoren und `:hover` / `:focus`, Box-Modell, `box-sizing`, `position`, Flexbox, Grid, Klassennamen mit Präfix |
| **05** | Responsive Design | Media Query, Mini-CSS-Reset, `order` |
| **06** | TypeScript Basics | `const` / `let`, Template-Strings, `Number()` / `NaN`, Union mit `null`, Arrays mit `find` / `filter` / `map`, `===`, **`type` statt `interface`**, Union und Literal Types, Funktionen als Parameter, `for … of` |
| **07** | TypeScript Advanced | Projektvorlage, Änderungen am Leistungsnachweis (07/6), Module mit Named Exports, `import type`, Ternary, Generics bei `useState`, Destructuring |
| **08** | React Basics | Funktionskomponenten mit `type Props`, `children`, eine CSS-Datei pro Komponente, `useState`, StrictMode, State Flow und Lifting State Up, Listen mit `key`-Prop, Conditional Rendering und Styling, Events mit `ChangeEvent`, `<select>` |
| **09** | React Hooks | StrictMode, Setter mit Funktion, Lazy Initial Value, `useEffect` mit Dependency Array und Cleanup, `async` / `await` mit `try` / `catch`, `fetch`, Custom Hooks, `window.setInterval`, Polling, `useLocalStorage`, Projekt-Tipps (09/36–37) |
| **–** | nicht in den Folien | Express (in den Folien nur genannt, 01/4 und 02/4), Context, `unknown` mit `typeof` / `in`, `instanceof`, `throw new Error`, `randomUUID`, `Date.now()`, `role`-Attribute. Diese Stellen sind im Rundgang markiert und begründet. |

---

## Aufteilung für die Gruppe

Der Code hat drei Bereiche. Jedes Mitglied nimmt sich **zuerst** einen Bereich vor, wird darin «Expertin» bzw. «Experte» und erklärt ihn danach den anderen. **Wichtig:** Das ist nur die Reihenfolge fürs Lernen. Im Fachgespräch muss **jede Person alles** erklären können, die Fragen kommen zu beliebigen Stellen.

| Bereich | Dateien | Zentrale Fragen, die die Person zuerst beherrscht | Folien | Teil A |
|---|---|---|---|---|
| **1. Server und Spielregeln** | alle Dateien in `server/src/`, dazu `client/src/types/poker.ts` und `client/src/utils/poker.ts` | REST-Routen, Methoden und Statuscodes; wie die Stimmen geheim bleiben; Heartbeat und Karteileichen; Durchschnitt bei T-Shirt-Grössen; warum der Server Eingaben prüft; warum Types und Regeln zweimal da sind | 02, 03, 06 | A.2, A.3 (Types und Regeln) |
| **2. Client-Basis und Home-Seite** | `client/package.json`, `vite.config.ts`, `main.tsx`, `App.tsx`, `useLocalStorage.ts`, `UserContext.ts`, `UserProvider.tsx`, `api/sessions.ts`, `HomePage.tsx`, `NameForm.tsx`, `SessionListItem.tsx`, `ErrorMessage.tsx`, `LoadingIndicator.tsx`, `index.css`, `App.css`, `HomePage.css`, `NameForm.css`, `SessionListItem.css` | Ansichtswechsel mit State in `App`; Name mit Context und `useLocalStorage`; Polling der Liste mit Cleanup; wie `api/sessions.ts` Fehler meldet; Grid und Box-Modell | 04, 05, 07, 08, 09 | A.3 (bis Home-Seite) |

| Bereich | Dateien | Zentrale Fragen, die die Person zuerst beherrscht | Folien | Teil A |
|---|---|---|---|---|
| **3. Pokertisch** | `PokerTablePage.tsx`, `SessionNotFound.tsx`, `PokerRoom.tsx`, `PokerTable.tsx`, `ParticipantSeat.tsx`, `CardDeck.tsx`, `DeckSwitcher.tsx`, `RoundResult.tsx` und ihre CSS-Dateien | Beitreten im `useEffect`, Verlassen im Cleanup, StrictMode und `ignore`-Flag; Polling als Heartbeat; Klick auf eine Karte; Lifting State Up; `<select>` fürs Deck; Flexbox-Tisch | 04, 05, 08, 09 | A.3 (ab Pokertisch-Seite) |

**Vorschlag, wenn ihr mehr oder weniger als drei seid:** Zu zweit nimmt eine Person Bereich 1 und 2 (Server und Home gehören über `api/sessions.ts` zusammen), die andere Bereich 3. Zu viert teilen sich zwei Personen den Pokertisch: `PokerTablePage.tsx` mit Beitreten und Verlassen für die eine, `PokerRoom.tsx` mit Polling, Aktionen und den Tisch-Komponenten für die andere.

**So bringt ihr euch gegenseitig alles bei**

1. Nach Abend 1 erklärt jede Person ihren Bereich den anderen in 15 Minuten, mit dem Code auf dem Bildschirm.
2. Die Zuhörenden stellen danach je drei Fragen aus Teil C, die zu diesem Bereich passen.
3. Ab Abend 3 tauscht ihr: Jede Person erklärt einen Bereich, der **nicht** ihr eigener ist. Wer den Bereich vorbereitet hat, korrigiert.
4. Für den Datenfluss «Klick auf eine Karte» (B.4) braucht es alle drei Bereiche. Übt ihn so: Person 3 beginnt beim Button, Person 2 übernimmt bei `api/sessions.ts`, Person 1 erklärt Server und Store. Danach erzählt jede Person den ganzen Weg allein.

**TODO Gruppe: Hier eintragen, wer welchen Bereich zuerst vorbereitet.**

---

# Teil A – Rundgang durch alle Dateien

## A.0 Das Wichtigste in fünf Sätzen

1. Der **Server** (Node.js mit Express) hält alle Sessions in einem Array im Arbeitsspeicher und bietet dafür eine REST-API unter `/api/sessions` an.
2. Der **Client** (React) zeigt entweder die `HomePage` oder die `PokerTablePage`. Welche, entscheidet ein State in `App`: die aktive Session-ID (**Ansichtswechsel mit State**, 09/36).
3. Der Client fragt den Server regelmässig nach dem aktuellen Stand (**Polling** mit `window.setInterval` im `useEffect`, 09/30) und zeigt ihn an. Jede Klick-Aktion ist ein Request, dessen Antwort sofort angezeigt wird.
4. Der **Name** liegt in einem Context und wird mit dem Custom Hook `useLocalStorage` gespeichert (09/32). Nach einem Reload ist nur noch der Name da, man landet wieder auf der Home-Seite.
5. **Beitreten** passiert im `useEffect` der `PokerTablePage`, **Verlassen** in dessen Cleanup. Verschwindet jemand ohne Verlassen (Tab zu, Reload), entfernt ihn der Server nach 15–20 s, weil seine Polls fehlen (**Heartbeat**).

## A.1 Projektaufbau: zwei Projekte

Im Projektordner liegen **zwei eigene Projekte** nebeneinander, jedes mit eigenem `package.json` («jedes `package.json` ist ein Projekt», Folien Node.js, 03/10). Es gibt kein `package.json` im Projektordner selbst. Darum braucht es zum Entwickeln zwei Terminals.

| Ordner / Datei | Zweck | Wichtig zu wissen | Folien |
|---|---|---|---|
| `client/` | React-App | Aufgebaut wie die **Projektvorlage «scrum-poker-hwz»** aus dem Kurs (07/4): gleiche Scripts, gleiche Konfiguration, Ordner `src/components`, `src/hooks`, `src/types`. Dazu kommen `src/api`, `src/context` und `src/utils`, damit nicht alles in einem Ordner liegt (09/37). | 07, 08 |
| `client/package.json` | Scripts und Abhängigkeiten | `dev` (Vite), `build` (`tsc -b && vite build`), `lint` (oxlint), `preview`, `format` und `format:check` (Prettier). Abhängigkeiten nur `react` und `react-dom`, alles andere sind Werkzeuge (`devDependencies`) aus der Vorlage. `packageManager: pnpm@12.11.2`, `engines.node >=24`. | 03, 07 |
| `server/` | Express-API | eigenes Node-Projekt. Einzige Abhängigkeit zur Laufzeit ist `express` (09/37: «keine zusätzlichen Libraries ausser Express»). | 03 |
| `server/package.json` | Scripts | `dev` = `node --watch src/index.ts` (startet bei jeder Änderung neu), `start` = `node src/index.ts`, `build` = `tsc -b` (nur Typprüfung), dazu `lint`, `format`, `format:check` wie im Client. | 03 |
| `*/pnpm-lock.yaml` | genaue Versionen | pro Projekt eines. Gleiche Installation auf jedem Rechner. Nie von Hand ändern. | 03 |
| `*/.prettierrc` | Formatierung | `semi: false`, `singleQuote: true`: ohne Semikolons, einfache Anführungszeichen, so wie der Code in den Folien und in der Vorlage. In VS Code «Format on Save» einschalten (09/37). | 09 |
| `*/tsconfig*.json` | TypeScript | aus der Vorlage, dazu `strict: true`. Wichtig: `verbatimModuleSyntax` (darum `import type`, 07/12), `allowImportingTsExtensions` (Imports mit `.ts` / `.tsx` am Ende), `erasableSyntaxOnly` (nur TypeScript, das man einfach weglassen kann, siehe B.15). | 07 |
| `.gitignore` | was nicht ins Git gehört | `node_modules/`, `dist/`, `*.tsbuildinfo`, das Abgabe-ZIP | – |
| `README.md` | Kurzanleitung | | |
| `docs/FACHGESPRAECH.md` / `.pdf` / `.html` | dieser Leitfaden | | |

**Warum zwei Projekte und kein gemeinsamer Ordner für die Types?** Die Vorlage ist ein reines React-Projekt, der Server ist ein eigenes Node-Projekt. Ein gemeinsamer Ordner bräuchte einen pnpm-Workspace, der in den Folien nicht vorkommt. Darum stehen die Types der API und die Regeln (Karten pro Deck, Namensprüfung) **bewusst zweimal** im Code: im Client in `types/poker.ts` und `utils/poker.ts`, im Server in `server/src/types.ts` und `server/src/rules.ts`. Beide Seiten verweisen im Kommentar oben aufeinander. Wer eine Regel ändert, muss sie an beiden Orten ändern (Übung E.8).

Die Projektdokumentation (`DOKUMENTATION.pdf` / `.html`) wird neben dem Code-ZIP abgegeben, nicht darin. Im ZIP fehlen ausserdem die internen Ordner `scripts/` (Doku- und Abgabe-Scripts) und `deploy/` (Betrieb auf dem Server). Die App braucht sie nicht.

## A.2 `server/` – Express-API

Express kommt in den Folien 1–9 nur als Name vor (01/4, 02/4, 09/37), die Details sind für den zweiten Kursteil angekündigt. Die Aufgabe verlangt einen Express-Server, darum erklären wir hier

jede Zeile selbst.

**`server/src/index.ts`** – Startpunkt des Servers (`pnpm dev` / `pnpm start`).

| Zeile | Erklärung |
|---|---|
| 9 | `setInterval(removeInactiveParticipants, CLEANUP_INTERVAL_MS)` – alle 5 s Karteileichen entfernen (Kapitel 8 der Doku). |
| 11-13 | `app.listen(PORT, …)` startet den HTTP-Server auf Port 3000 und meldet «Scrum Poker API läuft auf …». |

**`server/src/app.ts`** – die Express-App. Das Starten (`listen`) macht `index.ts`, hier wird nur zusammengesteckt, was die App tut.

| Zeile | Erklärung |
|---|---|
| 4-8 | `import express, { type NextFunction, … }` – ein Default Export (`express`) und Types im selben Import, mit `type` markiert (07/11-12). |
| 12 | `export const app = express()` – die App. |
| 14-16 | Kommentar: Warum kein CORS nötig ist (Proxy beim Entwickeln, gleiche Adresse im Betrieb). |
| 19 | `app.use(express.json())` – **Middleware**: liest JSON-Bodies und legt sie in `req.body` ab. |
| 21 | `app.use('/api/sessions', sessionsRouter)` – alle Session-Routen hängen unter diesem Präfix. |
| 24-26 | Nur im Produktivbetrieb (`SERVE_CLIENT`): `express.static` liefert den gebauten Client aus `client/dist` aus. Das passt zu 02/6: Der Server liefert den Client-Code aus. Beim Entwickeln macht das Vite. |
| 29-31 | Middleware **nach** allen Routen: Hat keine Route gepasst, kommt die Anfrage hier an → 404 als JSON statt einer HTML-Seite. |
| 34-43 | **Error-Handler**, erkennbar an den **vier** Parametern `(err, req, res, next)`. `SyntaxError` kommt von `express.json()` bei kaputtem JSON → 400. Alles andere → 500; der Fehler wird nur in der Konsole ausgegeben, nie an den Client. |

**`server/src/config.ts`** – alle Einstellungen an einem Ort.

| Zeilen | Erklärung |
|---|---|
| 5-9 | `PORT`: Standard 3000, denn dorthin leitet der Proxy des Clients weiter (`client/vite.config.ts`). Ist die Umgebungsvariable `PORT` gesetzt (Live-Server), gilt sie. `let` und `if` statt eines verschachtelten Ausdrucks. `Number(…)` macht aus dem Text eine Zahl (06/10). |
| 12 | `CLEANUP_INTERVAL_MS = 5000` – wie oft aufgeräumt wird. |
| 16 | `PARTICIPANT_TIMEOUT_MS = 15000` – wer sich so lange nicht gemeldet hat, gilt als weg. |
| 21-22 | `CLIENT_DIST` (Pfad zum gebauten Client, relativ zu `server/`) und `SERVE_CLIENT` (nur bei `NODE_ENV=production`). |

**`server/src/types.ts`** – die Types des Servers (`type` statt `interface`, 06/22): `DeckType`, `CardValue`, `Participant`, `SessionState`, `SessionSummary` und die Bodies der API. Gleicher Inhalt wie `client/src/types/poker.ts` (siehe A.1, warum doppelt).

**`server/src/rules.ts`** – die Regeln, die der Server prüft.

| Zeilen | Erklärung |
|---|---|
| 8-11 | `DECK_CARDS` – die Karten pro Deck. Der Type `{ fibonacci: CardValue[]; tshirt: CardValue[] }` verlangt, dass beide Decks da sind. |
| 13 | `DECK_TYPES` – alle Deck-Namen als Liste, für die Prüfung in `validation.ts`. |

| Zeilen | Erklärung |
|---|---|
| 15 | `MAX_NAME_LENGTH = 30` |
| 19-28 | `validateName()` – gibt die Fehlermeldung zurück oder `null`, wenn der Name gültig ist. Gleich wie im Client, damit beide dieselbe Meldung zeigen. |

**`server/src/store.ts`** – Speicher und Spielregeln, ohne Express.

| Zeilen | Erklärung |
|---|---|
| 5 | `randomUUID` aus `node:crypto` (nicht in den Folien): erzeugt eine zufällige, weltweit eindeutige ID. Eine Zeile, darum keine eigene Zufallsfunktion. |
| 17-23 | `StoredParticipant`: so speichert **der Server** eine Person, mit echter Stimme und `lastSeen` (Zeitpunkt der letzten Abfrage in Millisekunden). Der Client bekommt diesen Type nie zu sehen. |
| 25-30 | `Session`: ID, Deck, `revealed` und die Teilnehmer. |
| 33 | `let sessions: Session[] = []` – **der ganze Speicher**: ein Array, die älteste Session zuerst. `let`, weil `deleteSession()` ein neues Array zuweist. Nach einem Neustart ist es leer. |
| 39-62 | `toSessionState()` macht aus der gespeicherten Session das, was der Client sehen darf. Zeile 46: Die Stimme wird nur mitgeschickt, wenn aufgedeckt ist **oder** es die eigene Stimme der fragenden Person ist. Sonst bleibt `vote` `null`. **Hier wird die Geheimhaltung gemacht.** |
| 64-73 | `listSessions()` – pro Session nur ID und Anzahl Personen, für die Liste auf der Home-Seite. |
| 75-91 | `createSession()` – 6 Zeichen einer zufälligen UUID als ID. Ist sie schon vergeben, wird neu gewürfelt (`while`). Neue Sessions starten mit Fibonacci und ohne Teilnehmer, `push` hängt sie hinten an. |
| 94-96 | `getSession()` mit `find` (06/16): gibt `undefined` zurück, wenn es die Session nicht gibt. |
| 98-100 | `deleteSession()` mit `filter`: ein neues Array ohne diese Session. |
| 103-110 | `findParticipant()` – Person in einer Session suchen, auch mit `find`. |
| 112-124 | `addParticipant()` – neue Person mit `randomUUID()` als ID, ohne Stimme, `lastSeen = Date.now()`. |
| 126-135 | `removeParticipant()` – `filter` ohne die Person. Danach `revealIfEveryoneVoted`: War sie die Letzte ohne Stimme, wird aufgedeckt. |
| 142-152 | `removeInactiveParticipants()` – sucht pro Session alle, deren `lastSeen` länger als 15 s her ist, und entfernt sie. Leere Sessions bleiben stehen. |
| 155-162 | `castVote()` – Stimme setzen (`null` = zurückziehen), dann prüfen, ob alle gewählt haben. |
| 165-175 | `revealIfEveryoneVoted()` – leere Session: nichts tun. Hat jemand noch keine Stimme: nichts tun (`return`). Sonst aufdecken. |
| 177-186 | `revealCards()`, `resetRound()` – aufdecken bzw. alle Stimmen löschen und wieder verdecken. |
| 188-195 | `changeDeck()` – gleiches Deck: nichts tun. Anderes Deck: wechseln und neue Runde, weil alte Karten im neuen Deck nicht gelten. |

**`server/src/validation.ts`** – prüft die Bodies.

Ein Body kommt von aussen und kann alles enthalten (auch Unsinn per `curl`). Darum ist er hier `unknown`. TypeScript lässt einen erst damit arbeiten, wenn man ihn geprüft hat. `unknown` und die Prüfung mit `in` stehen nicht in den Folien; ohne sie müssten wir dem Client blind vertrauen.

| Zeilen | Erklärung |
|---|---|
| 10-19 | `getJoinError()`. Zeile 11: Ist der Body überhaupt ein Objekt (`typeof body !== 'object'`), nicht `null` (in JavaScript ist `typeof null` auch `'object'`) und hat er ein Feld `name` (`'name' in body`)? Danach weiss TypeScript, dass `body.name` existiert. Zeile 18: Für den Namen gilt dieselbe Regel wie im Client (`validateName`). |
| 22-37 | `getVoteError()` – `null` ist erlaubt (zurückziehen). Sonst muss der Wert ein Text sein, der im aktuellen Deck vorkommt (`includes`). |
| 40-50 | `getDeckError()` – das Deck muss einer der Namen aus `DECK_TYPES` sein. Die Schleife vergleicht mit `===`. |

Jede Funktion gibt die **Fehlermeldung** zurück oder `null`, wenn alles stimmt. Die Route macht daraus eine 400-Antwort.

**`server/src/routes/sessions.ts`** – alle Routen. Folien: HTTP-Methoden und Statuscodes (02/10-13).

Jede Route liest sich gleich: Session holen → gibt es sie nicht, 404 → Body prüfen (400) → Store ändern → antworten.

| Zeilen | Route | Erklärung |
|---|---|---|
| 4-5 | Kommentar zu `req.body` | Express gibt `req.body` den Type `any`. Erst nach der Prüfung weisen wir ihn einer Variable mit unserem Type zu. |
| 31 | `Router()` | Ein Express-Router sammelt Routen, `app.ts` hängt ihn unter `/api/sessions` ein. |
| 36-38 | `GET /` | Liste aller Sessions, Status 200 |
| 41-45 | `POST /` | Session erstellen → `201 { id }` |
| 48-56 | `DELETE /:sessionId` | 404, wenn unbekannt, sonst löschen → `204` ohne Body (`res.status(204).end()`) |
| 59-81 | `GET /:sessionId` | Zustand eines Tisches. `req.query.participantId` (Zeilen 66-69) ist nur ein Text, wenn er in der Adresse steht. Zeilen 74-79: **Heartbeat**, `lastSeen` wird aktualisiert. Eine unbekannte ID ist kein Fehler. |
| 84-101 | `POST /:sessionId/participants` | Beitreten. Zeilen 90-94: Body prüfen. Zeile 96: `const joinRequest: JoinSessionRequest = req.body` – ab hier ist der Body geprüft. Zeile 98: neue Person mit dem Namen ohne Leerzeichen am Rand. Antwort `201 { participantId }`. |
| 104-113 | `DELETE /:sessionId/participants/:participantId` | Verlassen, **idempotent**: auch eine schon entfernte Person ergibt 204. |
| 116-145 | `PUT /:sessionId/participants/:participantId/vote` | Prüft der Reihe nach: Session (404), Person (404), schon aufgedeckt (409), Karte gültig (400). Antwort mit der eigenen Stimme (Zeile 143). |
| 148-163 | `PUT /:sessionId/deck` | Deck prüfen (400), wechseln, neuen Zustand schicken |
| 166-174 | `POST /:sessionId/reveal` | aufdecken |
| 177-185 | `POST /:sessionId/reset` | neue Runde |

`req.params.sessionId` ist der Teil der Adresse an der Stelle von `:sessionId`.

## A.3 `client/` – React-App

### Einstieg und Konfiguration

**`client/index.html`** – die einzige HTML-Seite (aus der Vorlage). Zeile 6: `viewport`, damit Handys die echte Breite verwenden und die Media Queries greifen. Zeile 10: `<div id="root">`, hier hängt React die App hinein. Zeile 11 lädt `main.tsx`. Folien: 04/6.

**`client/vite.config.ts`** – aus der Vorlage, unverändert bis auf den Kommentar.

| Zeilen | Erklärung |
|---|---|
| 6 | `plugins: [react()]` – Vite versteht damit TSX (08/10-12). |
| 10-12 | **Proxy**: Jede Anfrage an `/api` geht an `http://localhost:3000`, also an Express. Der Browser spricht nur mit `localhost:5173`, darum braucht es kein CORS. Im Code steht deshalb immer nur `/api/…`. |

**`client/src/main.tsx`** – Startpunkt, wie in der Vorlage plus `UserProvider`. Folien: 09/3.

| Zeilen | Erklärung |
|---|---|
| 6 | `index.css` wird **vor** `App` geladen. So kommen die Styles der Komponenten danach und gewinnen bei gleich starken Regeln (z. B. der rote Button «Löschen»). |
| 9-10 | Named Imports mit Dateiendung, z. B. `import { App } from './components/App.tsx'` (07/10). |
| 16 | `createRoot(document.getElementById('root')!)` – das `!` sagt TypeScript: «dieses Element gibt es sicher» (es steht in `index.html`). Sonst wäre der Type `HTMLElement \| null`. Genau so in 09/3. |
| 17-21 | Verschachtelung: `StrictMode` → `UserProvider` → `App`. Der Provider liegt aussen, damit jede Komponente den Namen lesen kann. |

**`client/src/components/App.tsx`** – Wurzelkomponente (liegt wie in der Vorlage in `components/`). **Kern der Pflicht «Ansichtswechsel mit State»** (07/6, 09/36).

| Zeilen | Erklärung |
|---|---|
| 11 | `useState<string \| null>(null)` – die aktive Session-ID. `null` = Startseite. Der Type in spitzen Klammern ist ein Generic (07/16): Im State darf ein Text oder `null` liegen. |
| 13-15 | `leaveSession()` setzt den State zurück → wieder die Startseite. |
| 17-20 | **Conditional Rendering** (08/37): Standard ist die `HomePage`, mit gesetzter ID die `PokerTablePage`. Die ID geht per Props nach unten (`sessionId`), die Callbacks `onJoin` und `onLeave` melden nach oben (State Flow, 08/25-27). `onJoin={setActiveSessionId}` gibt den Setter direkt weiter. |
| 22-29 | Kopfzeile und `<main>` mit der gewählten Seite. Das leere `<>…</>` ist ein Fragment (08/38). |

### Types und Regeln im Client

**`client/src/types/poker.ts`** – alle Types des Clients, im Vorlagen-Ordner `types/`. Folien: 06.

| Zeilen | Was | Erklärung |
|---|---|---|
| 4 | `type DeckType = 'fibonacci' \| 'tshirt'` | **Union aus Literal Types** (06/23). Nur diese zwei Werte sind erlaubt, ein Tippfehler wie `'tshrit'` fällt schon beim Kompilieren auf. |
| 6 | `type CardValue = string` | Kartenwerte sind Texte (`'5'`, `'XL'`, `'?'`). Ob ein Wert gültig ist, prüft der Server (gehört er zum aktiven Deck?). |
| 8-11 | `Deck` | Ein Deck hat einen Namen für die Anzeige und eine Liste von Karten. |
| 14-17 | `TshirtSize` | eine Grösse mit ihrer Zahl für den Durchschnitt |

| Zeilen | Was | Erklärung |
|---|---|---|
| 21-28 | `Participant` | Was der Client über eine Person sieht: `hasVoted` immer, `vote` nur nach dem Aufdecken oder bei der eigenen Stimme. `CardValue \| null` zwingt den Code, den Fall «keine Stimme» zu behandeln. |
| 30-35 | `SessionState` | Der ganze Zustand eines Tisches. Antwort auf das Polling und auf alle Aktionen. |
| 38-41 | `SessionSummary` | ein Eintrag der Liste auf der Home-Seite: ID und Anzahl Personen |
| 46-69 | `JoinSessionRequest` und die anderen Bodies | Pro Route mit eigener Form ein Type, z. B. `CastVoteRequest` mit `value: CardValue \| null`. Der Client baut damit den Body, der Server erwartet dieselbe Form. |
| 73-80 | `RoundResult` | Ergebnis von `computeResult()` |

**`client/src/utils/poker.ts`** – die Regeln des Spiels im Client.

| Zeilen | Was | Erklärung |
|---|---|---|
| 16-25 | `DECKS` | Der Type `{ fibonacci: Deck; tshirt: Deck }` erzwingt, dass beide Decks da sind und jedes `label` und `cards` hat. |
| 28 | `DECK_TYPES` | Alle Deck-Namen als Liste. Daraus macht `DeckSwitcher` die `<option>`-Elemente. |
| 32-38 | `TSHIRT_SIZES` | Zahl pro Grösse (XS=1 … XL=8) für den Durchschnitt, eine Liste von Objekten. |
| 42 | `MAX_NAME_LENGTH = 30` | gleicher Wert wie im Server |
| 46-55 | `validateName()` | Gibt die Fehlermeldung zurück oder `null`, wenn der Name gültig ist. Prüft den Namen ohne Leerzeichen am Rand (`trim()`). Der Text mit `${MAX_NAME_LENGTH}` ist ein Template-String (06/9). |
| 58-72 | `toNumber()` | T-Shirt: Grösse mit `find` suchen (06/16), gibt es sie nicht (`?`), dann `null`. Sonst `Number(card)`: `'?'` und `'☕'` ergeben `NaN` → `null` (zählt nicht). |
| 75-86 | `closestTshirtSize()` | Startet mit der ersten Grösse und merkt sich die mit dem kleinsten Abstand. Das `<=` in Zeile 81 wählt bei Gleichstand die spätere, also grössere Grösse. |
| 89-125 | `computeResult()` | Erst Konsens (91-96), dann Summe und Anzahl der Zahlenkarten (99-107), ohne Zahlenkarte kein Durchschnitt (109-116), dann auf eine Nachkommastelle runden (119), bei T-Shirt die nächste Grösse (121-123). Gleiche Stimmen ergeben immer dasselbe Ergebnis, ohne Nebenwirkungen. |

### Name: Custom Hook und Context

**`client/src/hooks/useLocalStorage.ts`** – **Custom Hook** wie in der Hooks-Übung (09/26, 09/32). Erfüllt die Pflicht «Benutzername im localStorage» (09/36).

| Zeilen | Erklärung |
|---|---|
| 7-10 | `ReturnValue` – der Hook gibt ein Objekt mit `value` und `setValue` zurück (wie `useToggle` in 09/28). |
| 12 | `export function useLocalStorage(key: string)` – ein Hook ist eine normale Funktion, deren Name mit `use` beginnt und die selbst Hooks benutzt. |
| 14-20 | **Lazy Initial Value** (09/8): `useState(() => …)` – die Funktion läuft nur beim **ersten** Render, `localStorage` wird also nur einmal gelesen. Gibt es noch nichts (`null`), ist der Startwert `''`. |
| 23-26 | `setValue()` setzt den State **und** speichert sofort. Jeder Tastendruck ist gespeichert, auch nach einem Reload. |

**`client/src/context/UserContext.ts`** – Context für den Namen. **Context ist nicht in den Folien.** Die Aufgabenstellung verlangt ihn für den Namen, darum ist er da (siehe B.8).

| Zeilen | Erklärung |
|---|---|
| 6-9 | Was im Context steht: der Name ( `name` ) und eine Funktion zum Ändern ( `setName` ). |
| 12-15 | `createContext(…)` mit Startwert. Der Startwert gilt nur, wenn **kein** Provider darüber liegt; in der App kommt das nicht vor. |

**`client/src/components/UserProvider.tsx`** – füllt den Context. Zeile 13: `useLocalStorage('scrum-poker:name')`, der Präfix `scrum-poker:` verhindert Kollisionen mit anderen Apps auf derselben Adresse. Zeilen 17-21: `<UserContext value={…}>` – seit React 19 ist der Context selbst der Provider. Alles in `children` kann den Wert mit `useContext(UserContext)` lesen.

### Alle Anfragen an den Server

**`client/src/api/sessions.ts`** – eine Funktion pro API-Route. Komponenten rufen nie selbst `fetch()` auf. Folien: 09/17, 09/22-23.

| Zeilen | Erklärung |
|---|---|
| 21-38 | `send()` – die einzige Stelle mit `fetch()`. `body: string \| null = null` ist ein Default-Parameter: ohne Body einfach weglassen. Mit Body kommt der Header `Content-Type: application/json` dazu. **Wichtig:** `fetch()` wirft nur, wenn gar keine Antwort kommt (Server aus, kein Netz). Dann wird daraus «Server nicht erreichbar.» (Zeilen 35-37). Eine 404- oder 500-Antwort ist für `fetch()` kein Fehler. |
| 41-60 | `createError()` – macht aus einer Fehlerantwort ein `Error` mit Text. 502/503/504 kommen vom Vite-Proxy, wenn Express nicht läuft → auch «Server nicht erreichbar.». Sonst der Text aus `{ error }` vom Server, und wenn die Antwort kein JSON ist, «Anfrage fehlgeschlagen (HTTP …)». `new Error(…)` und `throw` sind nicht in den Folien; so bekommt jeder Fehler eine deutsche Meldung, die die Komponenten anzeigen. |
| 66-73 | `listSessions()` – das Muster aller Funktionen wie in 09/23: senden, `response.ok` prüfen (Status 200–299), sonst Fehler werfen, sonst `const dto = await response.json()` zurückgeben. Den Type der Antwort legt der Rückgabetyp `Promise<SessionSummary[]>` fest. |
| 76-83 | `createSession()` gibt nur die neue ID zurück. `const dto: CreateSessionResponse` sagt, welche Form die Antwort hat. |
| 86-95 | `deleteSession()` – 404 zählt als Erfolg: jemand anderes hat die Session schon gelöscht, das Ziel ist erreicht. |
| 100-116 | `getSession()` – schickt die eigene `participantId` in der Adresse mit (Heartbeat). 404 → `null`: «diese Session gibt es nicht». |
| 120-138 | `joinSession()` – schickt den Namen als `JoinSessionRequest`. 404 → `null`, sonst die neue `participantId`. |
| 141-209 | `leaveSession()`, `castVote()`, `changeDeck()`, `revealCards()`, `resetRound()` – alle nach demselben Muster. |
| 213-218 | `getErrorMessage()` – im `catch` ist der Fehler vom Type `unknown`. Mit `instanceof Error` (nicht in den Folien) prüfen wir, ob er eine `message` hat. |

### Home-Seite

**`client/src/components/HomePage.tsx`** – Startseite. Folien: 08, 09/13-15, 09/30.

| Zeilen | Erklärung |
|---|---|
| 20 | `SESSION_LIST_POLL_INTERVAL_MS = 3000` – die Liste wird alle 3 s neu geladen. |
| 22-25 | `type Props` mit dem Callback `onJoin` (von `App`). |
| 27 | `export function HomePage({ onJoin }: Props)` – Destructuring der Props in der Parameterliste (07/20). |

| Zeilen | Erklärung |
|---|---|
| 30-32 | Name aus dem Context. Ob er gültig ist (`validateName(name)`), wird **bei jedem Render berechnet**, nicht gespeichert (abgeleiteter Wert, 08/28). |
| 35-38 | Vier States. `useState<SessionSummary[] \| null>(null)`: `null` heisst «noch keine Antwort». |
| 41 | `isLoading` – berechnet: Wir laden, solange weder Liste noch Fehler da sind. |
| 44-74 | **Polling-Effekt** (gleiches Muster wie in `PokerRoom`): `let ignore = false` (46), innere `async function loadSessions()` mit `try` / `catch` (48-61), sofort einmal laden und dann alle 3 s mit `window.setInterval` (63-67), **Cleanup** `ignore = true` und `window.clearInterval(intervalId)` (70-73). Leeres Dependency Array `[]` (74): Der Effekt läuft beim Mount, das Cleanup beim Unmount. |
| 55-60 | Bei einem Fehler bleibt die alte Liste stehen, nur `setListError` wird aufgerufen. Der nächste Poll versucht es von selbst wieder. |
| 76-89 | `handleCreate()` – Ladezustand an, Session erstellen, dann `onJoin(sessionId)`: `App` wechselt zum Pokertisch. Bei Fehler Meldung und Button wieder frei. |
| 93-104 | `handleDeleted()` – entfernt die gelöschte Session sofort aus der Liste. `setSessions((currentSessions) => …)` mit einer Funktion (09/7) bekommt den neusten Stand, auch wenn inzwischen ein Poll angekommen ist. `filter()` erzeugt eine **neue** Liste, der alte State wird nicht verändert (08/33). |
| 119 | `NameForm` bekommt Name, Fehlermeldung und `setName` als Props. |
| 120-127 | Button «Neue Session erstellen», gesperrt bei ungültigem Namen oder während des Erstellens. Der Text wechselt mit einem Ternary (07/15). |
| 134-166 | Conditional Rendering mit `&&`: Fehler, Ladeanzeige, leere Liste, Hinweis, Liste. Zeile 158: `key={session.id}` – stabile ID statt Index (08/31). Zeile 161: `onJoin` wird an jeden Eintrag weitergegeben. |

**`client/src/components/NameForm.tsx`** – das Namensfeld. Folien: 08/41.

| Zeilen | Erklärung |
|---|---|
| 9-14 | Props: `name`, `error` (`null` = gültig), `onNameChange` (Callback nach oben). |
| 17-19 | `updateName(event: ChangeEvent<HTMLInputElement>)` – genau wie in 08/41: `event.target.value` ist der neue Text. |
| 23-25 | `<label htmlFor="name-input">` gehört zum Feld mit `id="name-input"`. Ein Klick auf das Label setzt den Cursor ins Feld. |
| 28-32 | **Conditional Styling** (08/36): Bei einem Fehler bekommt das Feld zusätzlich die Klasse `name-form__input--invalid` (roter Rand). |
| 34-35 | **Kontrolliertes Eingabefeld**: `value` kommt von oben, jede Eingabe geht über `onChange` zurück in den Context. |
| 36 | `maxLength` – der Browser lässt nicht mehr als 30 Zeichen zu. |
| 39 | Meldung nur, wenn es einen Fehler gibt (`error !== null`). |

**`client/src/components/SessionListItem.tsx`** – ein Eintrag der Liste.

| Zeilen | Erklärung |
|---|---|
| 10-17 | Props: die Session, ob Beitreten erlaubt ist, `onJoin` und `onDeleted` (beides Callbacks nach oben). |
| 26-27 | Eigener State pro Eintrag (`isDeleting`, `deleteError`): Nur dieser eine Eintrag zeigt «Wird gelöscht…». |
| 29-32 | Text «1 Person» / «x Personen». |
| 34-56 | `handleDelete()` – `window.confirm` (nicht in den Folien, eine Zeile) fragt nach. Bei Erfolg `onDeleted(session.id)`: die Seite entfernt den Eintrag aus der Liste (Lifting State Up). |

| Zeilen | Erklärung |
|---|---|
| 65-72 | «Beitreten» ruft nur `onJoin(session.id)` auf. Die Arrow Function im `onClick` gibt die ID dieses Eintrags mit. Beigetreten wird erst in der `PokerTablePage`. |
| 73-80 | «Löschen» mit eigenem Ladetext, gesperrt mit `disabled={isDeleting}`. |

**`client/src/components/ErrorMessage.tsx`** – rote Meldung als `<p>` mit `role="alert"` (Zeile 11): Screenreader lesen sie sofort vor. Die Komponente gibt es so ähnlich auch in 08/37.

**`client/src/components/LoadingIndicator.tsx`** – Ladeanzeige als Text mit `role="status"` (Zeile 12): Screenreader lesen sie vor, ohne zu unterbrechen. Kein drehender Kreis.

### Pokertisch-Seite (Beitreten und Verlassen)

**`client/src/components/PokerTablePage.tsx`** – die Seite, die `App` bei gesetzter Session-ID zeigt (Name wie in 08/49). Folien: 09/13-15, 09/24.

| Zeilen | Erklärung |
|---|---|
| 12-16 | Props: `sessionId` (von `App`) und `onLeave` (setzt den State in `App` zurück). |
| 21 | Name aus dem Context (`useContext(UserContext)`). Er ist hier immer gültig, weil auf der Startseite sonst nichts geht. |
| 23-25 | Drei States: eigene `participantId`, `notFound`, `joinError`. |
| 30-71 | **Join-Effekt.** Beitreten beim Mount, Verlassen im Cleanup beim Unmount. |
| 34-35 | `ignore` (Muster aus der React-Dokumentation) und `joinedId` (wer ist beigetreten, damit das Cleanup weiss, wen es abmelden muss). |
| 37-59 | `async function join()`: `joinSession(...)` (39). Ist der Effekt schon aufgeräumt (`ignore`, 40-47), wird der eben angelegte Teilnehmer gleich wieder abgemeldet. `null` → «nicht gefunden» (48-51). Sonst ID merken und anzeigen (52-53). Fehler → Meldung (54-58). |
| 61 | `join()` wird aufgerufen. Der Callback von `useEffect` selbst darf nicht `async` sein (09/23), darum die innere Funktion (09/24). |
| 63-70 | **Cleanup**: `ignore = true`; war man beigetreten, Leave-Request schicken. Ein Fehler ist egal (`.catch(() => {})`), weil der Heartbeat-Timeout die Person sowieso entfernt. |
| 71 | Dependency Array `[sessionId, name]`. |
| 73-101 | `renderContent()` entscheidet mit `if` und frühem `return` (08/7), was angezeigt wird: «nicht gefunden», Fehler mit «Zurück zur Übersicht», Laden oder der Tisch. |

**`client/src/components/SessionNotFound.tsx`** – Meldung «Diese Session existiert nicht (mehr)» mit `role="alert"` und dem Button «Zurück zur Übersicht» (Prop `onBack`, Zeilen 14-20). Benutzt von `PokerTablePage` (Beitreten schlägt fehl) **und** `PokerRoom` (Session wird gelöscht, während man am Tisch sitzt): derselbe Fehlerzustand.

### Pokertisch

**`client/src/components/PokerRoom.tsx`** – der Tisch eines beigetretenen Teilnehmers. Hält den ganzen Zustand des Tisches. Folien: 08/25-28, 09/30.

| Zeilen | Erklärung |
|---|---|
| 25 | `POLL_INTERVAL_MS = 1500` – muss deutlich unter dem Server-Timeout von 15 s liegen, weil jeder Poll als Heartbeat zählt. |
| 27-31 | Props: Session-ID, eigene ID, Callback `onLeave`. |
| 36-41 | State: `session` (Zustand vom Server, die einzige Quelle der Wahrheit), `pollError`, `notFound`, `isBusy` (eine Aktion läuft), `actionError`. |

| Zeilen | Erklärung |
|---|---|
| 44-80 | **Polling-Effekt**, gleiches Muster wie auf der Home-Seite. |
| 53-59 | Session gelöscht ( `null` ): `window.clearInterval` stoppt das Polling, damit nicht alle 1.5 s ein unnötiger Request mit 404 rausgeht. Dann `notFound` . |
| 62-67 | Fehler: der letzte Stand bleibt sichtbar, `setPollError` setzt nur die Meldung. |
| 71-72 | Erst den Timer anlegen, dann sofort laden. So kennt `loadSession` die `intervalId` schon, wenn die erste Antwort kommt (sie kommt erst nach dem `await` ). |
| 76-79 | Cleanup: `ignore = true` und `window.clearInterval` . Ohne das liefe das Polling nach dem Verlassen weiter und hielte uns als Teilnehmer am Leben. |
| 80 | Dependency Array `[sessionId, participantId]` . |
| 85-95 | `runAction(action)` – bekommt eine **Funktion als Parameter** (06/31), z. B. `() => revealCards(sessionId)` . Für alle vier Aktionen gleich: `isBusy` an, alten Fehler weg, die Funktion aufrufen und auf die Antwort warten, sie mit `setSession` sofort anzeigen; bei Fehler Meldung; `isBusy` aus. |
| 97-106 | Frühe Returns: nicht gefunden, noch nichts geladen (Laden oder Fehler). |
| 108-125 | `me` – die eigene Person mit `find` suchen. Fehlt sie ( `undefined` ), hat der Server uns entfernt → «Du bist nicht mehr am Tisch» mit «Zurück zur Übersicht». |
| 127-131 | Abgeleitete Werte: eigene Stimme, Anzahl Stimmen mit `filter` . Nicht im State, sondern bei jedem Render berechnet (08/28). |
| 134-143 | `roundStatus` – die Tischmitte: vor dem Aufdecken «x von y haben gewählt», danach `RoundResult` . Ein `if` statt eines verschachtelten Ausdrucks im JSX. |
| 145-152 | `handleCardClick()` – Klick auf die schon gewählte Karte → `null` (zurückziehen), sonst die Karte. |
| 154-164 | Die anderen Aktionen ( `handleDeckChange` , `handleReveal` , `handleReset` ), jede eine Zeile. |
| 169-173 | `DeckSwitcher` bekommt das Deck, `disabled` und `handleDeckChange` . |
| 176-182 | «Verlassen» ruft **nur** `onLeave` auf. `App` zeigt dann die Home-Seite, die `PokerTablePage` verschwindet, und ihr Cleanup schickt den Leave-Request. |
| 190-218 | `PokerTable` bekommt die Tischmitte als `children` : Stand oder Ergebnis, «Wird gesendet …» während einer Aktion, die Buttons «Karten aufdecken» (gesperrt, solange niemand gewählt hat) und «Neue Runde». |
| 220-225 | `CardDeck` bekommt Deck, eigene Stimme und `handleCardClick` . Gesperrt, wenn aufgedeckt ist oder eine Aktion läuft. |

**`client/src/components/PokerTable.tsx`** – die Plätze. Folien: 08/17, 08/31.

| Zeilen | Erklärung |
|---|---|
| 9-14 | Props mit `children: ReactNode` (08/17). |
| 18-20 | Die erste Hälfte sitzt oben, die zweite unten ( `Math.ceil` rundet auf, `slice` kopiert einen Teil der Liste). |
| 25-32 | ein `ParticipantSeat` pro Person mit `key={participant.id}` . |
| 34 | `{children}` – was `PokerRoom` mitgibt, landet in der Tischmitte. |
| 36-47 | Die untere Reihe wird nur gerendert, wenn jemand darin sitzt ( `bottomRow.length > 0 &&` ). |

**`client/src/components/ParticipantSeat.tsx`** – ein Platz. Folien: 08/36.

| Zeilen | Erklärung |
|---|---|
| 14-18 | Standard ist die leere Karte. Hat jemand gewählt und ist noch nicht aufgedeckt: verdeckte Karte ( `--covered` ). |
| 20-23 | Nach dem Aufdecken: Karte mit Wert ( `--revealed` ). Drei CSS-Klassen, keine Animation. |
| 26 | Conditional Styling: Der eigene Platz bekommt zusätzlich `seat--own` (Rahmen in Farbe). |
| 28-31 | Name und beim eigenen Platz das Abzeichen «Du» ( `isOwn &&` ). |

**`client/src/components/CardDeck.tsx`** – die eigenen Karten. Kein eigener State. Folien: 08/31, 08/36, 08/40.

| Zeilen | Erklärung |
|---|---|
| 7-13 | Props: `deck` , `selected` (eigene Stimme oder `null` ), `disabled` , `onCardClick` . |
| 20 | `DECKS[deck].cards.map(...)` – eine Karte pro Wert des aktiven Decks. |
| 23 | `key={value}` – die Werte sind im Deck eindeutig. |
| 25-29 | Conditional Styling mit Ternary: die gewählte Karte bekommt `playing-card--selected` . |
| 31 | `onClick={() => onCardClick(value)}` – die Arrow Function gibt den Wert dieser Karte mit. |

**`client/src/components/DeckSwitcher.tsx`** – Auswahl des Decks als `<select>` (08/42).

| Zeilen | Erklärung |
|---|---|
| 8-12 | Props: aktives Deck, `disabled` , `onChange` . |
| 16-24 | `updateDeck(event: ChangeEvent<HTMLSelectElement>)` – `event.target.value` ist nur ein `string` . Die Schleife sucht den passenden Eintrag in `DECK_TYPES` ; so weiss TypeScript, dass es wirklich ein `DeckType` ist, und es braucht keinen Cast. |
| 28-30 | `<label htmlFor="deck-select">` gehört zum `<select id="deck-select">` . |
| 31-37 | Kontrolliertes `<select>` : `value={deck}` kommt vom Server-Zustand. Wechselt jemand anderes das Deck, springt die Auswahl beim nächsten Poll mit. |
| 38-42 | ein `<option>` pro Deck mit `key` . |

**`client/src/components/RoundResult.tsx`** – Ergebnis nach dem Aufdecken. Folien: 08/28.

| Zeilen | Erklärung |
|---|---|
| 15-24 | Eine `for … of` -Schleife sammelt die Stimmen. Wer nicht gewählt hat ( `vote` ist `null` ), wird nicht mitgezählt. `?` und `☕` werden gezählt, um sie unten zu erwähnen. |
| 25 | `computeResult()` aus `utils/poker.ts` . Das Ergebnis ist kein State, sondern wird bei jedem Render neu berechnet. |
| 28-31 | Ohne Durchschnitt ein Strich `'–'` , mit `if` statt eines Ausdrucks im JSX. |
| 40-44 | Konsens hervorgehoben (gelber Kasten, Klasse `round-result__consensus` ). |
| 45-54 | `<dl>` : Beschreibungsliste für «Durchschnitt» und «Stimmen» (nicht in den Folien, passt aber genau: Begriff und Wert). |
| 55-60 | T-Shirt: Erklärung, wie die Grösse berechnet wurde. |

## CSS

Jede Komponente hat ihre **eigene CSS-Datei** mit gleichem Namen daneben, z. B. `CardDeck.tsx` → `CardDeck.css` , importiert oben in der Komponente (08/19). Die Klassennamen haben den Namen der Komponente als Präfix ( `card-deck__title` , `playing-card--selected` ), wie in 04/41 und

08/19. Farben und Abstände stehen als feste Werte in Pixeln, eine Farbtabelle steht als Kommentar oben in `index.css`.

| Datei | Inhalt | Folien |
|---|---|---|
| `client/src/index.css` | global, aus der Vorlage: Box-Modell `*, *::before, *::after { box-sizing: border-box; }` (Mini-CSS-Reset), Schrift, Überschriften, `:focus`-Rahmen, die Buttons (`.button`, `.button--secondary`, `.button:disabled`). | 04/14, 04/19-21, 05/8 |
| `App.css` | Kopfzeile und Inhaltsbereich (max. 960 px, zentriert), ab 768 px mehr Rand | 04/19 |
| `HomePage.css` | **Grid**: eine Spalte auf dem Handy, ab 768 px zwei Spalten 2 : 3 (`grid-template-columns`). Liste als Flexbox-Spalte. | 04/37-44, 05/7 |
| `SessionListItem.css` | Eintrag mit Flexbox und `flex-wrap`, roter Button «Löschen» (`.button--danger`) | 04/24-35 |
| `NameForm.css` | Feld mit `width: 100%`, roter Rand bei `--invalid` | 04/19 |
| `PokerTable.css` | **Flexbox**-Tisch. Handy: Plätze als Liste, Tisch darunter (`order: 1`). Ab 768 px: Plätze oben und unten in Reihen, ovaler Tisch dazwischen (`border-radius: 999px`). | 04/24-35, 05/11 |
| `ParticipantSeat.css` | Platz: Karte links und Name rechts, ab 768 px Karte über dem Namen; drei Kartenarten | 04/24-35 |
| `CardDeck.css` | Handkarten mit `flex-wrap`; die gewählte Karte rutscht mit `position: relative; top: -8px` nach oben | 04/22 |
| `DeckSwitcher.css`, `PokerRoom.css`, `RoundResult.css`, `ErrorMessage.css`, `LoadingIndicator.css`, `SessionNotFound.css` | kleine Dateien für die jeweilige Komponente | 04 |

**Mobile First:** Die Grundregeln gelten für das Handy, `@media (min-width: 768px)` ergänzt die Änderungen für grössere Bildschirme. Die Folien zeigen das Beispiel mit `max-width` (05/7); beides ist eine Media Query, nur die Richtung ist umgekehrt.

---

# Teil B – Musterantworten auf die typischen Fragen

## B.1 «Zeigen Sie Komponente X. Welche Props hat sie, wer rendert sie?»

Vorgehen: Datei öffnen → Kommentar oben vorlesen (wozu, wer benutzt sie) → `type Props` zeigen → sagen, was jede Prop bedeutet → mit `Ctrl+Shift+F` nach `<Komponentenname` suchen und die Stelle zeigen, die sie rendert.

> **Beispiel `CardDeck`:** «`CardDeck` zeigt die eigenen Karten. Sie bekommt vier Props: `deck`, welches Deck aktiv ist; `selected`, die eigene Stimme oder `null`; `disabled`, ob die Karten gesperrt sind; und `onCardClick`, eine Funktion, die sie beim Klick mit dem Kartenwert aufruft. Gerendert wird sie von `PokerRoom` (`PokerRoom.tsx:220-225`). Sie hat keinen eigenen State: Welche Karte gewählt ist, kommt vom Server über `PokerRoom`.»

> **Beispiel `PokerTablePage`:** «Die `PokerTablePage` ist die Seite mit dem Tisch. Props: `sessionId` und `onLeave`. Gerendert wird sie von `App`, sobald dort eine Session-ID im State steht (`App.tsx:17-20`). Sie tritt beim Mount der Session bei und verlässt sie im Cleanup. Steht die `participantId` fest, rendert sie `PokerRoom` mit dem eigentlichen Tisch.»

Alle Komponenten haben weniger als acht Props (höchstens vier, z. B. `SessionListItem`), so wie in 08/48 empfohlen. Die Tabelle aller Komponenten steht in der Dokumentation, Kapitel 3.2.

## B.2 «Warum liegt dieser State hier?»

Grundregel (Lifting State Up, 08/26): **State liegt in der untersten Komponente, die ihn ändern muss, und gerade so weit oben, dass alle, die ihn lesen, darunter liegen.**

| State | Ort | Satz zum Merken |
|---|---|---|
| aktive Session-ID | `App` | Entscheidet, welche Seite zu sehen ist. Darum liegt er über beiden Seiten (09/36). |
| Name | Context um die ganze App | Home **und** Tisch brauchen ihn, gespeichert mit `useLocalStorage`. |
| Session-Zustand | `PokerRoom` | Alle Teile des Tisches brauchen ihn; `PokerRoom` ist der gemeinsame Elternteil. |
| `participantId` | `PokerTablePage` | Gehört zum Beitreten und Verlassen, das dort im `useEffect` passiert. |
| `isDeleting` | `SessionListItem` | Betrifft nur diesen einen Eintrag. |
| eigene Stimme | **kein** eigener State | Steht schon im Server-Zustand (`me.vote`). Doppelt gespeichert könnte sie veralten. |
| Ergebnis, Gültigkeit des Namens | **kein** State | Wird bei jedem Render berechnet (08/28: möglichst viel ableiten). |

## B.3 «Wie wechselt die App zwischen Home und Pokertisch?»

> «Über State, so wie es der Leistungsnachweis seit der Änderung in 07/6 verlangt. In `App.tsx`, Zeile 11, liegt `activeSessionId`. Ist er `null`, zeigt `App` die `HomePage`, sonst die `PokerTablePage` mit der ID als Prop (Zeilen 17-20, Conditional Rendering). Die `HomePage` bekommt den Setter als Callback `onJoin`. Nach ‹Neue Session erstellen› ruft sie `onJoin` mit der neuen ID auf (`HomePage.tsx:82`, `onJoin(sessionId)`), beim Klick auf ‹Beitreten› ruft `SessionListItem` dasselbe mit der ID des Eintrags auf. Der Button ‹Verlassen› ruft `onLeave` auf, das setzt den State wieder auf `null`. Einen Router brauchen wir nicht, er ist nur noch eine optionale Erweiterung. Darum gibt es auch keinen Link auf eine Session, und nach einem Reload ist man wieder auf der Home-Seite.»

## B.4 «Verfolgen Sie einen Klick auf eine Karte.»

Mit dem Code daneben, in dieser Reihenfolge zeigen (Details: Dokumentation, Kapitel 6):

1. `CardDeck.tsx:31` – `onClick={() => onCardClick(value)}` meldet den Wert nach oben.
2. `PokerRoom.tsx:145-152` – `handleCardClick`: schon gewählt → `null`, sonst der Wert. Dann `runAction` mit einer Funktion, die `castVote` aufruft.
3. `PokerRoom.tsx:85-95` – `runAction`: `isBusy` an, alles gesperrt.
4. `api/sessions.ts:155-171` – `castVote`: `PUT /api/sessions/:id/participants/:pid/vote` mit `{ "value": "5" }`.

5. `vite.config.ts:10-12` – der Proxy leitet an Port 3000 weiter.
6. `routes/sessions.ts:116-145` – Route prüft: Session (404), Person (404), aufgedeckt (409), Karte (400).
7. `store.ts:155-175` – Stimme speichern, alle gewählt? → aufdecken.
8. `routes/sessions.ts:143` + `store.ts:46` – Antwort mit der eigenen Stimme, die anderen bleiben `null`.
9. `PokerRoom.tsx:90` – `setSession(newSession)` → React rendert neu → Karte hervorgehoben.
10. Die anderen sehen die verdeckte Karte bei ihrem nächsten Poll (≤ 1.5 s).

### B.5 «Welche API-Routen gibt es, welche Methode, warum?»

> «Alle unter `/api/sessions` in `server/src/routes/sessions.ts`. `GET` zum Lesen: die Liste und der Zustand eines Tisches. `POST` erstellt etwas Neues, dessen ID der Server vergibt: eine Session oder einen Teilnehmer, Antwort 201. `PUT` setzt einen Wert vollständig: die eigene Stimme und das Deck; zweimal derselbe Request ergibt dasselbe (idempotent). `DELETE` entfernt: Session oder Teilnehmer, Antwort 204 ohne Body. Aufdecken und Neue Runde sind Aktionen, darum `POST`. Fehler kommen immer als `{ error: '…' }`: 400 bei ungültiger Eingabe, 404 wenn etwas nicht existiert, 409 wenn man nach dem Aufdecken noch abstimmen will.»

Die Methoden und Statuscodes kommen in den Folien Architektur und Technologien vor (02/10-13). Die ganze Tabelle steht in der Dokumentation, Kapitel 5.

### B.6 «Was passiert, wenn der Server beim Polling nicht antwortet?»

> «`fetch` wirft, weil keine Antwort kommt. `send()` in `api/sessions.ts` (Zeilen 35-37) macht daraus den Fehler ‹Server nicht erreichbar.›. Über den Vite-Proxy kommt stattdessen eine 502, die `createError()` in denselben Text übersetzt. Im Polling-Effekt von `PokerRoom` landet der Fehler im `catch` (`PokerRoom.tsx:62-67`): `setPollError` setzt die Meldung, `session` wird aber **nicht** gelöscht. Darum bleibt der Tisch mit dem letzten Stand sichtbar, darüber steht ‹… Neuer Versuch läuft …›. Das Intervall läuft weiter; sobald der Server wieder antwortet, setzt der nächste Poll `pollError` auf `null`. Startet der Server neu, sind die Daten weg, die nächste Antwort ist 404, also `null`, und wir zeigen ‹Diese Session existiert nicht (mehr)›. Die Home-Seite macht es genauso.»

### B.7 «Warum ist dieser Type so definiert? Was ginge ohne ihn schief?»

- **`DeckType = 'fibonacci' | 'tshirt'`:** Mit `string` könnte man `'tshrit'` schreiben, und es fiele erst zur Laufzeit auf. Die Union aus Literal Types lässt nur die zwei Werte zu (06/23).
- **`vote: CardValue | null`:** `null` heisst «keine Stimme» bzw. «verdeckt». TypeScript zwingt jeden, der `vote` benutzt, an diesen Fall zu denken, z. B. `participant.vote !== null` in `ParticipantSeat.tsx:20`.
- **`StoredParticipant` vs. `Participant`:** Der Server speichert die echte Stimme und `lastSeen`, der Client bekommt nur, was er sehen darf. Zwei Types machen sichtbar, dass hier etwas verborgen wird.
- **`body: unknown` in `validation.ts`:** Von aussen kann alles kommen. Mit `unknown` erlaubt TypeScript keinen Zugriff, bevor wir mit `typeof` und `in` geprüft haben. Mit `any` würde TypeScript gar nichts prüfen. `strict: true` verbietet zudem ein stilles (implizites) `any`.
- **`type` statt `interface`:** Beides beschreibt die Form eines Objekts. Die Folien empfehlen `type` (06/22), und 09/37 verlangt, es einheitlich zu machen. Darum steht überall `type`,

auch bei den Props (`type Props`, wie in 08/15).

### B.8 «Welches Problem löst Context? Wann sind Props besser?»

> «Den Namen brauchen die `HomePage` (Eingabe) und die `PokerTablePage` (Beitreten). Ohne Context müsste `App` den Namen halten und an beide Seiten geben, wie in den Folien beim Lifting State Up (08/26). Das ginge bei uns auch, es sind nur zwei Ebenen. Die Aufgabenstellung verlangt für den Namen aber einen Context, darum legt `main.tsx` den `UserProvider` einmal um alles, und jede Komponente liest den Namen mit `useContext(UserContext)`. Context ist nicht in den Folien, wir haben ihn aus der React-Dokumentation. Props sind besser für alles andere: Sie zeigen, woher die Daten kommen, und bei häufigen Änderungen wie dem Tisch-Zustand alle 1.5 s rendert nur der betroffene Teil neu. Context lohnt sich für wenige Werte, die selten wechseln und die viele Stellen brauchen.»

### B.9 «Warum braucht es ein Cleanup im `useEffect`?»

> «Ein Effekt startet etwas, das ausserhalb von React weiterläuft: ein Intervall oder eine Anmeldung am Server. Wird die Komponente entfernt (Unmount), muss das wieder aufhören (09/15). Bei uns gibt es drei Cleanups:
>
> 1. **Polling der Liste** (`HomePage.tsx:70-73`): `window.clearInterval`, sonst liefen nach dem Wechsel zum Tisch weiter Requests für die Liste. Die Folien nennen das ein Memory Leak (09/30).
> 2. **Polling am Tisch** (`PokerRoom.tsx:76-79`): `window.clearInterval`, sonst liefe das Polling nach dem Verlassen weiter und würde uns als Teilnehmer am Leben halten.
> 3. **Beitreten** (`PokerTablePage.tsx:63-70`): Leave-Request, sonst sässe man nach dem Verlassen noch bis zum Timeout am Tisch.
>
> Dazu kommt das `ignore`-Flag: Eine Antwort, die erst nach dem Cleanup ankommt, schreibt nicht mehr in den State.»

### B.10 «Was macht StrictMode mit Effects?»

> «StrictMode (`main.tsx:17`) startet in der Entwicklung jeden Effekt beim ersten Anzeigen zweimal: Effekt, Cleanup, Effekt (08/24, 09/3). So fällt sofort auf, wenn ein Cleanup fehlt. Beim Beitreten gehen dadurch zwei Join-Requests los. Das Cleanup des ersten Durchlaufs setzt dessen `ignore` auf `true`. Kommt seine Antwort, sieht der Code in `PokerTablePage.tsx:40-47`: ‹ich bin schon aufgeräumt›, und meldet den eben angelegten Teilnehmer sofort wieder ab. Übrig bleibt der Teilnehmer des zweiten Durchlaufs. Im gebauten Produktivcode läuft der Effekt nur einmal.»

### B.11 «Was passiert bei einem Reload am Tisch?»

> «Man landet wieder auf der Home-Seite, nur der Name ist noch da. So verlangt es der Leistungsnachweis seit 07/6. Die aktive Session-ID ist ein normaler State in `App` und nach dem Reload wieder `null`. Der Name kommt aus dem `localStorage` (`useLocalStorage`). Bei einem Reload läuft kein React-Cleanup, auf dem Server sitzt man also noch am Tisch. Weil von diesem alten Platz keine Polls mehr kommen, entfernt ihn der Server nach höchstens 20 Sekunden (Heartbeat-Timeout). Wer wieder mitmachen will, tritt über die Liste neu bei und bekommt eine neue `participantId`.»

## B.12 «Kennt ihr Custom Hooks? Warum gibt es keinen Hook fürs Polling?»

> «Ja, `useLocalStorage` in `client/src/hooks/` ist ein Custom Hook, genau wie in der Übung aus Vorlesung 9 (09/32). Er kapselt `useState` und das Speichern im `localStorage`, und der `UserProvider` benutzt ihn. Fürs Polling haben wir bewusst keinen Hook gebaut. Es gibt zwei Stellen, und beide machen nach einer Antwort etwas anderes (die Liste behält bei Fehlern den alten Stand, der Tisch stoppt bei einer gelöschten Session das Intervall). Ein gemeinsamer Hook bräuchte dafür Parameter und Callbacks und wäre schwerer zu lesen als zwei kurze `useEffect`. So wie `useAutoCounter` in 09/30 sieht jedes Polling bei uns aus: `window.setInterval` im Effekt, `window.clearInterval` im Cleanup. Statt Requests abzubrechen (`AbortController`, nicht Kursstoff), ignorieren wir späte Antworten mit dem `ignore`-Flag.»

## B.13 «Warum habt ihr keine Error Boundary und keine Tests mit einer Test-Library?»

> «In den Projekt-Tipps steht ‹keine Class-Components› und ‹keine zusätzlichen Libraries ausser Express und optional React Router› (09/37). Eine Error Boundary gibt es in React nur als Klasse, darum haben wir sie weggelassen. Fehler von Server-Aufrufen fangen wir überall mit `try`/`catch` ab und zeigen eine Meldung. Ein Fehler beim Rendern wäre ein Programmierfehler; dagegen helfen `strict` in TypeScript, `pnpm lint` und das Ausprobieren. Test-Libraries wie Vitest wären zusätzliche Libraries und kommen in den Folien nicht vor. Wir prüfen mit `pnpm lint`, `pnpm format:check` und `pnpm build` (dabei läuft die Typprüfung) in beiden Ordnern und klicken jeden Ablauf in zwei Browserfenstern durch, auch auf Handy-Breite in den DevTools.»

## B.14 «Warum zwei Projekte, und warum stehen manche Regeln doppelt?»

> «Der Client ist die Projektvorlage aus dem Kurs (07/4), ein reines React-Projekt. Den Server haben wir als eigenes Node-Projekt daneben gelegt, wie in den Folien zu Node.js: ein `package.json` pro Projekt (03/10). Ein gemeinsamer Ordner für die Types bräuchte einen Workspace, den wir sonst nirgends erklären könnten. Darum stehen die Types der API und zwei Regeln zweimal da: die Karten pro Deck und die Namensprüfung (`client/src/utils/poker.ts` und `server/src/rules.ts`). Der Nachteil ist, dass man bei einer Änderung beide Stellen anpassen muss. Darauf weist der Kommentar oben in beiden Dateien hin. Vergisst man es, merkt man es sofort: Der Server lehnt dann die Karte mit 400 ab.»

## B.15 «Wie läuft der Server mit TypeScript, ohne dass ihr ihn kompiliert?»

> «Node.js 24 kann `.ts`-Dateien direkt ausführen. Es entfernt beim Laden einfach die Type-Angaben (Type Stripping) und führt den Rest als JavaScript aus. Darum steht in `server/package.json` nur `node --watch src/index.ts`. Das klappt, weil wir nur TypeScript benutzen, das man weglassen kann: Types, `type`-Imports, Type-Angaben an Variablen. Die Einstellung `erasableSyntaxOnly` in der `tsconfig.json` prüft genau das. `pnpm build` im Server macht nur die Typprüfung mit `tsc`, erzeugt aber keine Dateien. Im Client übersetzt Vite das TypeScript.»

---

# Teil C – Fragenkatalog

| # | Frage | Kurze Antwort |
|---|---|---|
| 1 | Was ist der Unterschied zwischen Props und State? | Props kommen von aussen und sind in der Komponente nur lesbar (08/16). State gehört der Komponente, ändert sich mit der Setter-Funktion und löst ein neues Rendern aus. |
| 2 | Wie kommt die aktive Session-ID zur `PokerTablePage`? | Sie liegt als State in `App` und geht per Prop `sessionId` nach unten. Gesetzt wird sie über den Callback `onJoin`, zurückgesetzt über `onLeave`. |
| 3 | Was heisst «Callback via Props»? | Die Eltern-Komponente gibt eine Funktion als Prop nach unten, das Kind ruft sie auf, um etwas nach oben zu melden (State Flow, 08/25-27). Beispiel: `onCardClick`, `onJoin`, `onDeleted`. |
| 4 | Warum darf man State nicht direkt ändern? | React merkt Änderungen nur über den Setter mit einem **neuen** Wert. Direkt verändert, rendert nichts neu. Darum `filter()` beim Löschen, das eine neue Liste erzeugt (08/33). |
| 5 | Wozu dient die `key`-Prop in Listen? | Damit React beim Ändern der Liste weiss, welches Element welches ist (08/31). Wir nehmen stabile IDs (`session.id`, `participant.id`), nie den Index. |
| 6 | Was macht `useEffect`, und wann läuft er? | Er führt Code **nach** dem Rendern aus, für alles ausserhalb von React (Server, Timer). Er läuft beim Mount und immer dann, wenn sich ein Wert im Dependency Array ändert. `[]` = nur beim Mount (09/13). |
| 7 | Warum ist der Callback im Effekt nicht direkt `async`? | Ein Effekt darf nur eine Cleanup-Funktion zurückgeben, eine `async`-Funktion gibt aber ein Promise zurück (09/23). Darum eine innere `async function` und sie sofort aufrufen (09/24). |
| 8 | Was macht das `ignore`-Flag? | Jeder Effekt-Durchlauf hat sein eigenes `ignore`. Das Cleanup setzt es auf `true`. Eine Antwort, die danach kommt, wird nicht mehr in den State geschrieben. |
| 9 | Warum ruft `loadSession` selbst `window.clearInterval` auf? | Wurde die Session gelöscht, ist weiteres Abfragen sinnlos. Sonst ginge alle 1.5 s ein Request mit 404 raus. |
| 10 | Was heisst `useState(() => …)`? | Die Funktion läuft nur beim ersten Render (Lazy Initial Value, 09/8). So wird `localStorage` nicht bei jedem Render gelesen. |
| 11 | Was heisst `setSessions((current) => …)`? | Der Setter bekommt eine Funktion und ruft sie mit dem neusten State auf (09/7). Sicher, auch wenn sich der State inzwischen geändert hat (z. B. ein Poll ist angekommen). |
| 12 | Was ist ein Custom Hook? | Eine Funktion, deren Name mit `use` beginnt und die selbst Hooks benutzt (09/26). Bei uns `useLocalStorage`. |
| 13 | Was bleibt nach einem Reload erhalten? | Nur der Name (`localStorage`). Die aktive Session-ID ist normaler State und wieder `null` (07/6). |
| 14 | Wo seht ihr den gespeicherten Namen im Browser? | DevTools → Application → Local Storage, Schlüssel `scrum-poker:name` (02/19-20). |
| 15 | Warum Polling und nicht WebSockets? | Einfacher, passt zu REST, kommt in den Folien vor (09/30). Nachteil: bis 1.5 s Verzögerung und mehr Requests. Für ein Team von 5–10 Personen kein Problem. |
| 16 | Warum 1.5 s am Tisch und 3 s bei der Liste? | Am Tisch soll es sich live anfühlen und der Heartbeat braucht Abstand zum 15-s-Timeout. Die Liste ist weniger dringend. |

| # | Frage | Kurze Antwort |
|---|---|---|
| 17 | Was ist CORS und warum braucht ihr es nicht? | Der Browser blockiert Antworten von einer anderen Adresse (auch ein anderer Port zählt), wenn der Server es nicht erlaubt. Der Vite-Proxy macht `/api` zur selben Adresse wie die Seite. |
| 18 | Was macht `express.json()`? | Middleware, die JSON-Bodies liest und in `req.body` ablegt. Bei kaputtem JSON wirft sie einen `SyntaxError` → unser Error-Handler → 400. |
| 19 | Was ist Middleware? | Eine Funktion `(req, res, next)`, die vor oder nach den Routen läuft. Sie kann die Anfrage verändern, beantworten oder mit `next()` weitergeben. |
| 20 | Woran erkennt Express den Error-Handler? | An den vier Parametern `(err, req, res, next)`. Darum steht `_next` da, obwohl er nicht benutzt wird. |
| 21 | Warum ist `app.ts` von `index.ts` getrennt? | `app.ts` beschreibt, was die App tut, `index.ts` startet sie und den Aufräum-Timer. So ist jede Datei kurz und hat eine Aufgabe. |
| 22 | Warum prüft der Server den Namen, obwohl der Client schon prüft? | Jeder kann mit `curl` Requests schicken. Prüfung im Client = Komfort, Prüfung im Server = Sicherheit. Beide benutzen dieselbe Regel `validateName`. |
| 23 | Warum 201 beim Erstellen und 204 beim Löschen? | 201 Created: etwas Neues ist entstanden. 204 No Content: Erfolg, aber es gibt nichts zurückzuschicken. |
| 24 | Wann kommt 409? | Wenn jemand nach dem Aufdecken noch abstimmen will. Die Anfrage ist richtig gebaut, passt aber nicht zum aktuellen Zustand. |
| 25 | Wie verhindert ihr, dass man Stimmen in den DevTools sieht? | Der Server schickt vor dem Aufdecken nur `hasVoted`, `vote` ist `null` (`store.ts:46`). Im Network-Tab sieht man das in der Antwort. |
| 26 | Warum sieht man die eigene Stimme vor dem Aufdecken? | Damit der Client die gewählte Karte aus dem Server-Zustand hervorheben kann, ohne sie selbst zu speichern. Für die eigene Person ist sie kein Geheimnis. |
| 27 | Wann wird automatisch aufgedeckt? | Wenn alle (mindestens eine Person) gewählt haben: nach einer Stimme oder wenn die letzte Person ohne Stimme geht. |
| 28 | Warum startet ein Deck-Wechsel eine neue Runde? | Alte Karten (z. B. `13`) gibt es im neuen Deck nicht, der Durchschnitt wäre sinnlos. |
| 29 | Wie zählen `?` und `☕`? | Nicht zum Durchschnitt, aber zur Anzahl Stimmen. Nur `?`/`☕` → Durchschnitt «–». |
| 30 | Was heisst Konsens bei euch? | Mindestens zwei Stimmen und alle gleich (auch zweimal `?`). |
| 31 | Unterschied `unknown` und `any`? | Beide nehmen alles an. Bei `unknown` verlangt TypeScript eine Prüfung vor der Nutzung, `any` schaltet die Prüfung ab. Beides kommt in den Folien nicht vor; wir benutzen `unknown` nur dort, wo Daten von aussen kommen. |
| 32 | Was macht `const joinRequest: JoinSessionRequest = req.body`? | `req.body` hat bei Express den Type `any`. Erst nach der Prüfung in `validation.ts` geben wir ihm unseren Type, damit TypeScript ab dort `joinRequest.name` kennt. |
| 33 | Was bedeutet das `!` in `getElementById('root')!`? | «Dieser Wert ist sicher nicht `null`». Das `<div id="root">` steht in `index.html`. So auch in 09/3. |
| 34 | `type` oder `interface`? | Beides beschreibt Objekte. Wir nehmen überall `type`, wie in den Folien empfohlen (06/22), und einheitlich (09/37). |
| 35 | Was macht `import type`? | Importiert nur einen Type, der beim Übersetzen ganz wegfällt. Mit `verbatimModuleSyntax` (aus der Vorlage) ist das Pflicht (07/12). |
| 36 | Was macht `async`/`await`? | `await` wartet auf ein Promise, ohne den Browser zu blockieren. Fehler fängt man mit `try`/`catch` (09/17). |

| # | Frage | Kurze Antwort |
|---|---|---|
| 37 | Warum wirft `fetch` bei 404 keinen Fehler? | `fetch` wirft nur, wenn keine Antwort kam. Jede HTTP-Antwort ist für `fetch` ein Erfolg. Darum prüfen wir `response.ok` bzw. `response.status` selbst (09/23). |
| 38 | Warum gibt `getSession` bei 404 `null` zurück statt eines Fehlers? | «Gibt es nicht» ist ein normaler Fall, kein Absturz. `null` lässt sich mit einem `if` prüfen, ohne `try` / `catch`. |
| 39 | Warum `.catch(() => {})` beim Leave-Request? | Ein Fehler beim Verlassen ist egal, der Heartbeat-Timeout entfernt die Person trotzdem. Ohne `.catch` gäbe es eine «Uncaught (in promise)»-Meldung in der Konsole. |
| 40 | Was ist ein kontrolliertes Eingabefeld? | Der Wert des Felds kommt aus dem State (`value`), jede Eingabe geht über `onChange` zurück. React ist die einzige Quelle für den Inhalt. Bei uns das Namensfeld und das `<select>`. |
| 41 | Wie liest ihr den Wert aus einem `<select>`? | Im Handler mit `ChangeEvent<HTMLSelectElement>` steht der gewählte Wert in `event.target.value` (08/42). Weil das ein `string` ist, suchen wir den passenden `DeckType` in `DECK_TYPES`. |
| 42 | Was macht `children` bei `PokerTable`? | Alles, was zwischen `<PokerTable>` und `</PokerTable>` steht, kommt als Prop `children` an (08/17). `PokerTable` zeigt es in der Mitte, ohne zu wissen, was es ist. |
| 43 | Was ist Conditional Rendering, was Conditional Styling? | Rendering: je nach Bedingung eine andere Komponente oder nichts (`&&`, Ternary, `if` mit `return`). Styling: je nach Bedingung eine andere CSS-Klasse (`className={… ? … : …}`), 08/36-37. |
| 44 | Warum hat jede Komponente eine eigene CSS-Datei? | So steht in 08/19: Styles liegen neben der Komponente, die sie braucht. Der Präfix im Klassennamen verhindert, dass sich Komponenten gegenseitig stören. |
| 45 | Was macht `box-sizing: border-box`? | `width` schliesst Padding und Rahmen ein (Box-Modell, 04/19-21). Elemente werden nicht unerwartet breiter. Bei uns der Mini-CSS-Reset oben in `index.css`. |
| 46 | Flexbox oder Grid – wann was? | Flexbox für eine Richtung (Reihe von Buttons, Karten, Plätzen), Grid für Spalten und Zeilen (Home-Seite). |
| 47 | Was heisst Mobile First? | Zuerst Regeln für kleine Bildschirme schreiben, mit `min-width`-Media-Queries für grössere ergänzen. |
| 48 | Wozu `role="alert"`, `role="status"` und `label` mit `htmlFor`? | Screenreader lesen Fehler sofort und Ladehinweise höflich vor; das Label gehört fest zu seinem Feld. Je ein Attribut, mehr Barrierefreiheit bauen wir nicht ein. |
| 49 | Was passiert, wenn zwei Personen gleichzeitig das Deck wechseln? | Der Server bearbeitet Requests nacheinander. Der zweite gewinnt, beide sehen beim nächsten Poll denselben Zustand. |
| 50 | Was passiert mit den Daten bei einem Neustart des Servers? | Alles weg (Array im Arbeitsspeicher). Bewusst so, es gibt keine Datenbank. |
| 51 | Warum verlasst ihr die Session nicht beim Schliessen des Tabs? | Beim Schliessen kommt nicht sicher noch ein Request an (Absturz, Akku leer, Netz weg). Darum verlassen wir uns auf den Heartbeat: Ein geschlossener Tab ist nach höchstens 20 s weg. |
| 52 | Was passiert mit zwei Tabs derselben Person? | Jeder Tab tritt selbst bei und bekommt eine eigene `participantId`: zwei Einträge am Tisch. |
| 53 | Wie startet man die App? | Zwei Terminals: im Ordner `server/` `pnpm dev` (Port 3000), im Ordner `client/` `pnpm dev` (Port 5173). Vorher in beiden einmal `pnpm install`. |

| # | Frage | Kurze Antwort |
|---|---|---|
| 54 | Was prüft `pnpm lint`? | oxlint (aus der Vorlage) sucht typische Fehler, z. B. fehlende Werte im Dependency Array. 09/37 verlangt, es auszuführen. |
| 55 | Wozu Prettier? | Formatiert den Code einheitlich (ohne Semikolons, einfache Anführungszeichen). `pnpm format:check` prüft, ob alles formatiert ist (09/37). |

---

# Teil D – Demo und Checkliste

## D.1 Demo-Drehbuch (max. 3 Minuten, zwei Browserfenster)

Vorbereitung: Fenster **A** (normal, Name «Anna») links, Fenster **B** (privates Fenster, damit `localStorage` getrennt ist) rechts. Beide auf http://localhost:5173. Keine Session offen.

| Zeit | Fenster | Aktion | Was ihr sagt |
|---|---|---|---|
| 0:00 | A | Name «Anna» steht schon da | «Der Name kommt aus dem `localStorage`, gelesen von unserem Hook `useLocalStorage`.» |
| 0:10 | B | Name leer lassen → Meldung, Buttons grau. Dann «Ben» eingeben | «`validateName` sperrt Erstellen und Beitreten. Der Server prüft mit derselben Regel.» |
| 0:25 | A | «Neue Session erstellen» | «`POST /api/sessions` → 201. Dann ruft die `HomePage` `onJoin` auf, und `App` setzt die Session-ID im State. Darum sehen wir jetzt die `PokerTablePage`.» |
| 0:40 | B | Session erscheint in der Liste (≤ 3 s) → «Beitreten» | «Die Liste pollt alle 3 Sekunden. ‹Beitreten› setzt denselben State in `App`.» |
| 0:55 | A | Karte **5** klicken | «`PUT …/vote`. Bei Ben erscheint nur eine verdeckte Karte, der Wert verlässt den Server nicht.» |
| 1:10 | B | Karte **8** klicken → automatisch aufgedeckt | «Alle haben gewählt → der Server deckt auf. Durchschnitt 6.5.» |
| 1:25 | A | «Neue Runde», beide wählen **3** | «Konsens wird hervorgehoben.» |
| 1:45 | B | Deck-Auswahl auf **T-Shirt**, A wählt S, B wählt L | «Ein `<select>`. Ein Deck-Wechsel startet für alle eine neue Runde. S=2, L=5, Durchschnitt 3.5 → M.» |
| 2:05 | B | Seite neu laden (F5) | «Nach einem Reload bleibt nur der Name, Ben ist wieder auf der Home-Seite. Sein alter Platz bekommt keine Polls mehr und verschwindet bei Anna nach höchstens 20 Sekunden (Heartbeat).» |
| 2:25 | A | DevTools → Network: alle 1.5 s ein `GET …?participantId=` | «Das Polling ist zugleich der Heartbeat.» |
| 2:40 | A | «Verlassen» → Home | «`onLeave` setzt den State in `App` zurück. Die `PokerTablePage` verschwindet, und ihr Cleanup schickt den DELETE.» |

Plan B, falls etwas hängt: Seite neu laden. Ist der Server abgestürzt: Terminal zeigen, im Ordner `server/` `pnpm dev` neu starten und dabei die Meldung «Server nicht erreichbar.» als Teil der Fehlerbehandlung erklären.

## D.2 Checkliste für den Tag

**Termine** (Folien Einführung, 01/5): Git-Repo-Link an den Dozenten **bis Ende November 2026**, Fachgespräch am **8. Dezember 2026** (letzte Vorlesung), Abgabe **bis 22. Dezember 2026**.

**Am Vortag**

- [ ] Node.js 24 und pnpm 12 installiert (`node -v`, `pnpm -v`)
- [ ] Projekt aktuell, in `client/` und in `server/` je `pnpm install`
- [ ] In beiden Ordnern `pnpm lint`, `pnpm format:check` und `pnpm build` ohne Fehler
- [ ] Demo einmal komplett mit Stoppuhr durchgespielt
- [ ] Jede Person kann Teil B ohne Unterlagen beantworten
- [ ] Laptop-Akku, Netzteil, ggf. Adapter für den Beamer

**30 Minuten vorher**

- [ ] Alle anderen Programme schliessen, Benachrichtigungen aus
- [ ] Terminal 1: `cd server`, `pnpm dev` → «Scrum Poker API läuft auf http://localhost:3000»
- [ ] Terminal 2: `cd client`, `pnpm dev` → Vite meldet http://localhost:5173
- [ ] Browser A und privater Browser B auf http://localhost:5173, Zoom 125 % für den Beamer
- [ ] Alte Test-Sessions löschen (oder den Server neu starten)
- [ ] Editor offen mit diesen Dateien in Tabs: `server/src/routes/sessions.ts`, `server/src/store.ts`, `client/src/components/App.tsx`, `client/src/api/sessions.ts`, `client/src/hooks/useLocalStorage.ts`, `client/src/components/HomePage.tsx`, `client/src/components/PokerTablePage.tsx`, `client/src/components/PokerRoom.tsx`, `client/src/utils/poker.ts`
- [ ] Zeilennummern im Editor eingeschaltet, Schrift gross genug
- [ ] Globale Suche (`Ctrl+Shift+F`), «Go to Line» (`Ctrl+G`) und «Go to Definition» (`F12`) geübt
- [ ] React Developer Tools im Browser installiert: Wo liegt der State, welche Props bekommt eine Komponente? (07/5)
- [ ] `DOKUMENTATION.pdf` offen als Rückfallebene

**Während des Gesprächs**

- Erst die Stelle im Code zeigen, dann erklären. Den deutschen Kommentar oben in der Datei vorlesen ist ein guter Einstieg.
- Ehrlich bleiben: «Das hat die KI so geschrieben, ich verstehe es so: …» ist besser als raten.
- Bei unbekannten Fragen laut denken und im Code nachsehen.

---

# Teil E – Übungsaufgaben zum Selbsttest

Jede Aufgabe: Vermutung aufschreiben → ändern → beide `pnpm dev` laufen lassen (Server startet bei Änderungen selbst neu, der Client lädt neu) → beobachten → zurücksetzen mit `git checkout .` (ohne Git, z. B. aus dem ZIP: vorher eine Kopie des Ordners anlegen). Die Lösung steht darunter, erst nachher lesen.

**1. Polling-Intervall** – Setze `POLL_INTERVAL_MS` in `PokerRoom.tsx:25` auf `20000`. Was passiert nach 20 s am Tisch?

> Der Server entfernt dich nach 15–20 s, weil der Heartbeat zu selten kommt. Beim nächsten Poll fehlst du in der Liste → «Du bist nicht mehr am Tisch». Zeigt: Polling und Timeout hängen zusammen.

**2. Cleanup weglassen** – Lösche in `HomePage.tsx:72` die Zeile mit `window.clearInterval`. Geh von der Home-Seite an einen Tisch und schau in den Network-Tab.

> Die Liste wird weiter alle 3 s geladen, obwohl die Home-Seite gar nicht mehr zu sehen ist. Das ist das Memory Leak aus 09/30. Mit StrictMode sind es sogar zwei Intervalle.

**3. StrictMode-Schutz entfernen** – Lösche in `PokerTablePage.tsx:43-45` den Block, der den überflüssigen Teilnehmer wieder abmeldet. Tritt einer Session bei.

> Im Dev-Modus sitzt du zweimal am Tisch (einmal als Karteileiche, die nach 20 s verschwindet). Zeigt, warum StrictMode Effekte doppelt startet: Der Fehler fällt sofort auf.

**4. Session-ID im Header** – Zeige in der Kopfzeile in `App.tsx:24-26` neben «Scrum Poker» die aktive Session-ID an, aber nur, wenn eine gesetzt ist.

> `{activeSessionId !== null && <span> · Session {activeSessionId}</span>}`. Der State liegt schon in `App`, also braucht es keine neue Prop. Gutes Beispiel für Conditional Rendering.

**5. State direkt ändern** – Ersetze in `HomePage.tsx:100-102` das `filter` durch `currentSessions.splice(currentSessions.findIndex((s) => s.id === deletedSessionId), 1)` und gib danach `currentSessions` zurück. Lösche eine Session.

> Der Eintrag bleibt stehen, bis der nächste Poll kommt. Der Setter bekommt dieselbe Liste (dieselbe Referenz) zurück, darum rendert React nicht neu (08/28, 08/33).

**6. Geheimhaltung brechen** – Ersetze in `store.ts:46` die Bedingung durch `true`.

> Vor dem Aufdecken stehen alle Stimmen in der Antwort (Network-Tab). Die Oberfläche zeigt sie trotzdem verdeckt. Das zeigt: Geheimhaltung muss auf dem Server passieren.

**7. Automatisch aufdecken** – Lösche in `store.ts:166-168` die Prüfung auf eine leere Session. Was passiert, wenn die letzte Person eine Session verlässt und danach jemand neu beitritt?

> Ohne Teilnehmer gibt es niemanden ohne Stimme, also wird die leere Session «aufgedeckt». Die nächste Person sieht einen aufgedeckten Tisch ohne Stimmen.

**8. Neue Karte** – Füge `'40'` ins Fibonacci-Deck in `utils/poker.ts:19` ein (nur im Client). Klicke die Karte an.

> Die Karte erscheint, aber der Server antwortet 400 «Diese Karte gibt es im aktuellen Deck nicht.», und die Meldung steht über dem Tisch. Erst wenn `'40'` auch in `rules.ts:9` steht, klappt es. Genau das ist der Preis der doppelten Regeln (B.14).

**9. Neuer Deck-Typ** – Ergänze in `types/poker.ts:4` den `DeckType` um `'powers'` (ohne `DECKS` anzupassen). Starte im Ordner `client/` `pnpm build`.

> TypeScript meldet Fehler überall, wo `DECKS[…]` steht (z. B. `CardDeck.tsx:20` mit `DECKS[deck]`): Für `'powers'` gibt es in `DECKS` keinen Eintrag. So zeigt TypeScript alle Stellen, die man noch

> anpassen muss.

**10. Statuscode** – Ändere in `routes/sessions.ts:131` den Code 409 zu 400. Was sieht man im Client, und warum ist 409 passender?

> Im Client ändert sich nichts, die Meldung kommt aus `{ error }`. 400 hiesse aber «die Anfrage ist falsch gebaut», dabei ist sie richtig und passt nur nicht zum Zustand der Runde.

**11. Fehler simulieren** – Stoppe während einer Runde nur den Server (`Ctrl+C` im Terminal mit `server/`), der Client läuft weiter.

> Meldung «Server nicht erreichbar. Neuer Versuch läuft …», der Tisch bleibt sichtbar (der Vite-Proxy antwortet mit 502). Server neu starten → Daten weg → «Diese Session existiert nicht (mehr)». Erkläre warum.

**12. Context ohne Provider** – Entferne `<UserProvider>` und `</UserProvider>` in `main.tsx`. Versuche, einen Namen einzugeben.

> Das Feld bleibt leer. Ohne Provider gilt der Startwert aus `UserContext.ts:12-15`: Name `''` und ein `setName`, das nichts tut.

**13. Unkontrolliertes `<select>`** – Lösche in `DeckSwitcher.tsx:34` die Zeile `value={deck}`. Wechsle in Fenster B das Deck und schau auf Fenster A.

> In A wechseln die Karten, aber die Auswahl zeigt weiter das alte Deck. Ohne `value` merkt sich der Browser die Auswahl selbst, und sie folgt dem Server-Zustand nicht mehr.

**14. Breakpoint** – Ändere in `HomePage.css:49` `768px` auf `1200px` und verkleinere das Fenster langsam. Wo springt das Layout um? Mit den DevTools (Mobile View, `Ctrl+Shift+M`) geht das auch.

**15. Box-Modell** – Entferne die Regel mit `box-sizing` in `index.css:13-17`. Wo wird etwas breiter?

> Das Namensfeld hat `width: 100%` plus Padding und Rahmen. Mit `content-box` wird es breiter als seine Karte und ragt über den Rand.

**16. Dritte Ansicht** – Baue eine Seite «Hilfe»: eine neue Komponente `HelpPage.tsx` mit eigener CSS-Datei, ein Button in der Kopfzeile und ein zweiter State in `App`, der entscheidet, ob die Hilfe zu sehen ist. Erkläre jeden Schritt.

**17. Neue API-Route** – Baue `GET /api/sessions/:sessionId/result`, die das Ergebnis einer aufgedeckten Session zurückgibt (sonst 409). Welche Methode, welche Statuscodes, welcher Type in welchen zwei Dateien? Prüfe sie mit `curl http://localhost:3000/api/sessions/<id>/result`.

**18. Lazy Initial Value** – Ersetze in `useLocalStorage.ts:14-20` die Funktion durch einen direkten Wert. Was ändert sich für den Benutzer, was im Hintergrund?

> Für den Benutzer nichts. Aber `localStorage.getItem` läuft jetzt bei jedem Render, also bei jedem Tastendruck, statt nur einmal (09/8).

**19. Datenfluss rückwärts** – Starte bei `RoundResult.tsx` und verfolge, woher `participants` kommt, bis zum Eintrag im Array `sessions` im Server.

**20. Zufällige Zeile** – Eine Person öffnet eine beliebige Datei und zeigt auf eine beliebige Zeile. Die andere erklärt: Was passiert hier? Warum steht das hier? Was ginge kaputt, wenn man die

Zeile löscht? So läuft das Fachgespräch ab.